A CONCISE STUDY & REFERENCE GUIDE

DATABASE MANAGEMENT SYSTEMS

Quick Theory · Key Diagrams · Exam-Focused Notes

CHAPTER 1 — INTRODUCTION

1.1 History of DBMS · Advantages & Limitations of DBMS · Users of DBMS

1.2 Software Modules in DBMS · Architecture of DBMS

Designed for OAVS Computer Teacher (TGT) · NVS · KVS · DSSSB · OSSSC · B.Ed Entrance · MCA · BCA · B.Tech · University & Competitive Examinations

SELF-CONTAINED QUICK-REVISION MATERIAL

Table of Contents

Introduction: Database & DBMS **3**
1.1 History of DBMS **4**
Advantages of DBMS **5**
Limitations of DBMS **6**
File System vs DBMS 6
Users of DBMS **7**
1.2 Software Modules in DBMS **9**
Architecture of DBMS **11**
One / Two / Three-Tier Architecture 11
Quick Revision Notes **14**
Exam Questions **15**

Introduction: Database & DBMS

WHAT IS A DATABASE?

A **database** is an organised, structured collection of related data stored electronically so that it can be easily accessed, managed, and updated. It represents facts about a specific subject — e.g., a college database stores student, course, and faculty data. A database by itself is just the stored data; the **DBMS** is the software used to manage and interact with it.

Key characteristics: it is **self-describing** (stores metadata along with data), supports **sharing** among multiple users/applications, maintains **relationships** between data items, and minimises duplication. It can hold text, numbers, images, audio or video, ranging from a small contact list to a large enterprise system like a bank's customer database.

DEFINITION

A **Database Management System (DBMS)** is system software that enables users to define, create, maintain, and control access to a database, acting as an interface between the database and users or application programs.

1.1 History of DBMS

Before DBMS existed, organisations stored data in manual paper records and later in simple computer files, with each application managing its own separate files. As data grew larger and needed to be shared across programs, this approach caused heavy data duplication, inconsistency, and weak security. To solve these problems, researchers developed structured data models — first hierarchical and network models in the 1960s, and then, most importantly, the **Relational Model** proposed by **Dr. E. F. Codd** at **IBM** in **1970**. This became the foundation of modern DBMS software, leading to the first commercial relational database by Oracle in 1979 and the rise of SQL-based systems throughout the 1980s and beyond.

Evolution of DBMS (Key Milestones)

- **1950s:** Manual / punch-card record keeping.
- **1960s:** File-based processing – each application kept its own separate files.
- **Mid 1960s:** Hierarchical Model – IBM's IMS, tree-like parent-child structure.
- **Late 1960s:** Network Model (CODASYL) – a record can have multiple parents.
- **1970: E. F. Codd** (IBM) proposed the **Relational Model** – data stored in tables based on set theory.
- **1979:** Oracle released the first commercial RDBMS.
- **1980s:** SQL standardised by ANSI; DB2, Sybase, SQL Server emerge; OODBMS developed.
- **2000s–Present:** Open-source (MySQL, PostgreSQL) and NoSQL/NewSQL databases for Big Data.

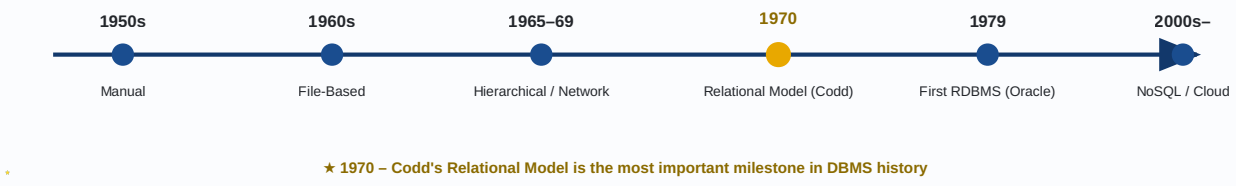


Fig. 1.1 — Evolution / History Timeline of DBMS

Exam Point

E. F. Codd proposed the Relational Model in **1970** at **IBM** — the single most frequently asked fact from this topic.

Advantages of DBMS

A DBMS was developed to overcome the shortcomings of file-processing systems. Key benefits:

- **Controls Data Redundancy:** centralised storage avoids duplicate data.
- **Data Consistency:** a single update reflects everywhere the data is used.
- **Data Sharing:** multiple users/programs can access the same database at once.
- **Data Integrity:** constraints (primary key, foreign key, not-null) ensure valid data.
- **Data Security:** authentication & authorisation restrict unauthorised access.
- **Backup & Recovery:** automatic recovery after crashes/failures.
- **Data Independence:** storage structure can change without altering application code.
- **Efficient Query Processing:** the Query Optimiser executes queries faster than manual search programs.
- **Reduced Development Time:** developers focus on logic; DBMS handles storage/security.
- **Concurrent Access Control:** locking/transactions allow safe simultaneous use.

Shortcut (Mnemonic)

"RISCS-BID" – **R**edundancy control, **I**ntegrity, **S**ecurity, **C**oncurrency, **S**haring, **B**ackup & recovery, **I**ndependence, **D**evelopment time reduced.

Limitations of DBMS

- **High Cost:** licensing, hardware and skilled-staff costs.
- **Increased Complexity:** more features make the software harder to manage.
- **Large Size / Resource Overhead:** requires significant disk space and memory.
- **Need for Specialised Staff:** trained DBAs are required.
- **Overhead for Simple Applications:** unnecessary for small, single-user tasks.
- **Migration Cost:** converting file systems to a DBMS is costly and time-consuming.

> ⚠ **Common Mistake**
>
> "Data redundancy" is **not** a limitation — controlling redundancy is an **advantage**. The real limitation is **cost and complexity**.

File System vs DBMS

Table 1.1 — File System vs DBMS

Basis	File System	DBMS
Redundancy	High	Controlled
Consistency	Hard to maintain	Maintained automatically
Data Sharing	Difficult	Easy
Security	OS-level only	Multi-level authentication
Integrity	Manual enforcement	Enforced via constraints
Backup/Recovery	Manual, unreliable	Automatic, reliable
Cost	Low	Higher
Best Suited For	Small, single-user apps	Large, multi-user enterprise apps

Users of DBMS

DEFINITION

Users of DBMS are the people who interact with the database, directly (queries, administration) or indirectly (through application programs).

Types of DBMS Users

DBMS users are classified based on their technical skill and how they interact with the database. Each category is given a different interface and level of access, matching what they need to do:

- **DBA (Database Administrator):** The "chief officer" of the database – defines the schema, sets storage structure and access methods, grants/revokes user privileges, and handles backup, recovery and performance tuning.
- **Database Designer:** Identifies entities, attributes and relationships, and designs the conceptual/logical schema before the database is built.
- **Naive / Parametric User:** Has little to no technical knowledge; interacts only through fixed forms, menus or pre-written application programs.
- **Sophisticated User:** Technically skilled (engineers, analysts, scientists) who write SQL queries directly to satisfy their own information needs.
- **Application Programmer:** Writes and maintains application software using host languages combined with embedded SQL or APIs such as JDBC/ODBC.
- **Casual User:** Accesses the database infrequently, often needing different information each time, typically via query tools or report generators.
- **Specialised User:** Develops advanced, non-traditional applications such as expert systems, CAD tools, or multimedia/knowledge-based systems.

Table 1.2 — Types of DBMS Users at a Glance

User Type	Technical Skill	Interaction Mode	Example
DBA	Very High	Full administrative control	College IT admin
Database Designer	High	Designs schema before development	ER-diagram designer
Naive / Parametric User	Low	Pre-written forms / menus, no SQL	Bank teller
Sophisticated User	High	Direct SQL queries	Business analyst
Application Programmer	High	Embedded SQL / APIs in code	Software developer
Casual User	Medium	Occasional queries / reports	Manager checking reports
Specialised User	Very High	Complex, non-traditional apps	CAD engineer

Remember

Naive users **never** write SQL directly – they use ready-made forms. Sophisticated users write SQL themselves. The **DBA** is the most important user category for exams.

1.2 Software Modules in DBMS

DEFINITION

The internal software modules of a DBMS are the components that accept requests, process queries, and manage physical data storage – broadly grouped into the **Query Processor** and the **Storage Manager**.

Query Processor

The **Query Processor** is the group of components that interprets the commands given by a user or application, checks their syntax, converts them into a form the system can execute, and works out the most efficient way to carry them out. It is essentially the "front-end" of the DBMS — everything a user types (DDL or DML statements) passes through it first, before any actual data is touched.

- **DDL (Data Definition Language) Interpreter:** interprets CREATE/ALTER/DROP statements.
- **DML (Data Manipulation Language) Compiler:** translates SELECT/INSERT/UPDATE/DELETE into internal form.
- **Query Optimiser:** selects the least-cost execution plan.
- **Query Evaluation Engine:** executes the low-level instructions produced by the optimiser to actually retrieve/modify data. *Example:* for `SELECT Name FROM Student WHERE RollNo = 5;`, it carries out the chosen plan — e.g., looking up Roll No. 5 using an index and returning the matching Name.

Storage Manager

The **Storage Manager** is the group of components that provides the interface between the low-level data stored on disk and the application programs/queries submitted to the system. It is responsible for the actual storage, retrieval, and updating of data, and for ensuring that this happens correctly, securely, and consistently even when many users access the database at once.

- **Authorisation & Integrity Manager:** checks permissions and constraints.
- **Transaction Manager:** ensures consistency and handles concurrent transactions (ACID).
- **File Manager:** manages disk space allocation and file structures.
- **Buffer Manager:** transfers data between disk and main memory.

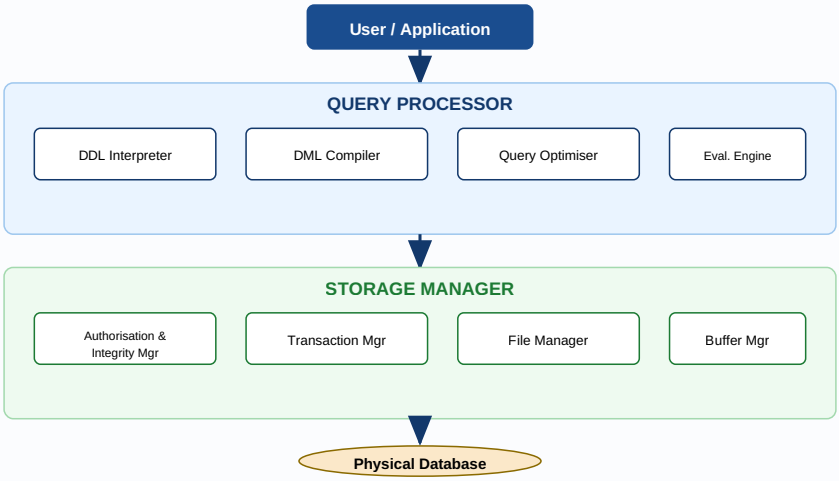


Fig. 1.2 — Internal Software Modules of a DBMS

Shortcut (Mnemonic)

Storage Manager – **"ATFB"**: **A**uthorisation, **T**ransaction, **F**ile, **B**uffer Manager.

Architecture of DBMS

DEFINITION

DBMS Architecture describes how the client, application logic, and database server are organised and communicate – classified as one-tier, two-tier, or three-tier.

The architecture chosen for a DBMS affects how many machines are involved, where business logic runs, how secure the database is from direct user access, and how well the system scales as the number of users grows. As applications evolved from single standalone programs to large web-based systems, DBMS architecture evolved correspondingly from one-tier to multi-tier designs.

One-Tier Architecture

Client, application and database all reside on a **single machine** – no network involved. Used for learning/ standalone apps.

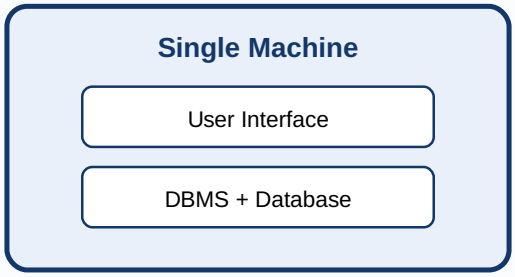


Fig. 1.3 — One-Tier Architecture

Example: *MS Access or SQLite running on a single laptop for personal use.* ***Advantage:*** *simple, fast, no network overhead.* ***Drawback:*** *not suitable for multiple users or remote access.*

Two-Tier Architecture

Split into **Client** (UI + logic) and **Server** (DBMS), connected directly via ODBC/JDBC. Called "fat client" architecture.



Fig. 1.4 — Two-Tier Architecture

Example: *a college desktop application where each staff computer (client) connects directly to a central database server over a LAN.* ***Advantage:*** *faster than three-tier for small networks.* ***Drawback:*** *client must be updated whenever business logic changes; database is directly exposed to clients.*

Three-Tier Architecture

Adds a middle **Application Tier** between client and database. Standard for modern web applications – improves security and scalability.

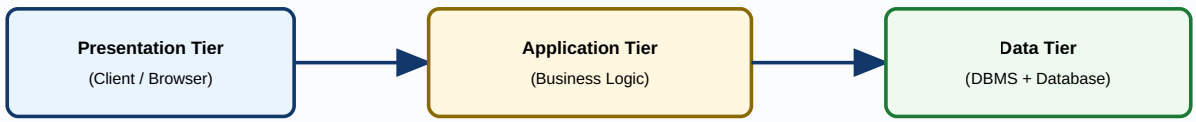


Fig. 1.5 — Three-Tier Architecture

Example: *an e-commerce website – the browser (presentation) talks to a web/app server (business logic like cart and pricing rules), which talks to the database server (data).* ***Advantage:*** *client never touches the database directly, so it is more secure and easier to scale by adding more application servers.* ***Drawback:*** *more complex to design, deploy and maintain than two-tier.*

Table 1.3 — Two-Tier vs Three-Tier Architecture

Basis	Two-Tier	Three-Tier
Structure	Client ↔ Server	Client ↔ App Server ↔ Database
Security	Lower	Higher (DB hidden behind app server)
Scalability	Limited	High
Business Logic	Mostly on the client ("fat client")	On the middle application server ("thin client")
Typical Use	Small LAN apps	Web / enterprise applications

Table 1.4 — Advantages & Limitations of Each Architecture

Architecture	Advantages	Limitations
One-Tier	Simple to set up; fast (no network delay); low cost; good for learning	Single user only; no remote access; not scalable
Two-Tier	Faster than three-tier for small networks; simpler to develop than three-tier; direct client-server communication	Database exposed directly to clients (less secure); client must be updated for logic changes; limited scalability
Three-Tier	High security (DB hidden behind app server); highly scalable; centralised business logic, easier maintenance	More complex to design and deploy; higher development/infrastructure cost; extra network hop can add latency

Remember

Two-tier = Client + Server (2 layers). Three-tier = Client + Application Server + Database Server (3 layers) — the standard for web apps.

Exam Point

As the number of tiers increases, **security and scalability improve**, but so does **system complexity and cost**. This trade-off is a common exam discussion point.

Quick Revision Notes

Advantages vs Limitations of DBMS

Advantages	Limitations
Controlled redundancy	High cost
Data consistency & integrity	Increased complexity
Data security	Large resource overhead
Backup & recovery	Needs skilled DBA/staff
Data independence	Overhead for simple apps

20 One-Line Revision Points

1. DBMS = software to define, create, maintain and control database access.
2. E. F. Codd proposed the Relational Model in 1970 at IBM.
3. Oracle released the first commercial RDBMS in 1979.
4. Hierarchical model = tree (parent-child); Network model = multiple parents (CODASYL).
5. NoSQL databases handle Big Data and unstructured content.
6. DBMS controls (not eliminates) data redundancy.
7. Data independence separates schema from application programs.
8. DBMS provides authentication & authorisation for security.
9. DBMS is costlier and more complex than a file system.
10. DBA is responsible for schema definition and access authorisation.
11. Naive users use only pre-written application programs; no SQL.
12. Sophisticated users write SQL queries directly.
13. Query Processor = DDL (Data Definition Language) Interpreter, DML (Data Manipulation Language) Compiler, Optimiser, Evaluation Engine.
14. Storage Manager = Authorisation, Transaction, File and Buffer Managers.
15. Data Dictionary stores metadata about the database.
16. One-tier: client, app and database on one machine.
17. Two-tier: client connects directly to the database server.
18. Three-tier: adds a middle application server; used for web apps.
19. Three-tier improves security by hiding the database from the client.
20. Query flow: Parser → Optimiser → Evaluation Engine → Storage Manager → Database.

Exam Questions

Key Theory Questions

1. Define DBMS. How is it different from a file-processing system?
2. Who proposed the Relational Model, and in which year?
3. List the advantages and limitations of DBMS.
4. Describe the types of DBMS users with examples.
5. Explain the internal software modules of a DBMS with a diagram.
6. Differentiate between two-tier and three-tier architecture with diagrams.

Quick MCQs

1. Father of the Relational Model? — **(b) E. F. Codd**
2. Year of the Relational Model paper? — **(c) 1970**
3. First commercial RDBMS released by? — **(c) Oracle**
4. Which user only uses pre-written forms? — **Naive user**
5. Which module chooses the best query execution plan? — **Query Optimiser**
6. Which architecture has a middle application server? — **Three-tier**

Exam Tips

Support answers with labelled diagrams, memorise "Codd – 1970" precisely, and present "differentiate between" answers in tabular form for full marks.